MediaPipe Vision Studio Report

1. Problem

Computer vision is used in many real-world applications such as face detection, hand tracking, pose analysis, gesture interaction, and background effects. However, many beginner-level computer vision projects require training a custom model before producing useful results. This increases the learning curve for students who want to understand practical computer vision workflows.

This project addresses that issue by building a simple web application that demonstrates multiple ready-to-use MediaPipe vision tasks in one place. The main goal is to allow users to upload or capture an image, select a computer vision task, and immediately view the processed output.

The supported tasks are:

- Face detection
- Hand tracking
- Rule-based gesture recognition
- Pose detection
- Background removal and background blur

2. Approach

The project is implemented as a Python Streamlit application. Streamlit was chosen because it allows developers to build interactive web interfaces quickly with minimal setup. This makes it suitable for an educational computer vision project.

The project is divided into two main files:

- `app.py` : responsible for the user interface, image input, task selection, and result display.
- `vision_tasks.py` : responsible for the MediaPipe and OpenCV processing functions.

The user can provide input in two ways:

- Uploading a local image file.
- Capturing an image using the browser camera.

After the image is loaded, it is converted into an RGB NumPy array. Based on the selected task, the image is passed to the corresponding processing function.

The main functions are:

- `detect_face()` : uses MediaPipe Face Detection to detect faces and draw bounding boxes.
- `detect_hands()` : uses MediaPipe Hands to detect hands and draw 21 hand landmarks.
- `recognize_gesture()` : uses MediaPipe Hands to detect a hand, estimate which fingers are open, and classify the gesture using simple rule-based logic.
- `detect_pose()` : uses MediaPipe Pose to detect body landmarks and draw the pose skeleton.

- `remove_background()`: uses MediaPipe Selfie Segmentation to separate the person from the background and apply a blur effect to the background.

The project uses MediaPipe pre-trained solutions instead of training a new machine learning model. This keeps the application lightweight and allows the focus to remain on understanding computer vision pipelines and how different vision tasks work.

3. Results

The application successfully provides a single dashboard for five different computer vision tasks. For each task, the user can view the original input image and the processed output side by side.

The expected results are:

- **Face Detection:** detected faces are highlighted using green bounding boxes.
- **Hand Tracking:** hand landmarks and hand connections are drawn on the detected hands.
- **Gesture Recognition:** one detected hand is annotated with landmarks and a gesture label such as `Fist`, `Open Palm`, `Victory / Peace`, `Thumbs Up / Like`, or `Pointing`.
- **Pose Detection:** body landmarks and skeleton connections are drawn on the person.
- **Background Removal:** the person is preserved while the background is blurred.

The application is useful as a learning demo because it shows the difference between several important computer vision concepts, including object detection, landmark detection, rule-based classification, and image segmentation.

4. Limitations

Although the project works as an educational demo, it has some limitations:

- The gesture recognition system is rule-based, not trained using a machine learning model.
- Gesture recognition may fail when the hand is rotated, partially hidden, or captured from an unusual angle.
- Thumb detection is simplified and may not work correctly for both left and right hands in all positions.
- The application processes still images and camera snapshots, not true real-time video streams.
- Detection quality depends on lighting, camera angle, image quality, and how clearly the face, hand, or body is visible.
- Background removal depends on a segmentation mask threshold, so edges around hair, arms, or clothing may not always be clean.
- The project does not currently include an automated test suite.
- The application is not yet deployed as a public web app.

5. Future Improvements

Several improvements can be added in future versions of the project:

- Add real-time webcam processing using `streamlit-webrtc`.
- Improve gesture recognition using handedness information and better geometric rules.
- Add more gestures such as OK sign, rock sign, stop sign, and custom classroom gestures.
- Display confidence scores and detection statistics in the user interface.

- Add a download button to save the processed output image.
- Add custom background replacement instead of only background blur.
- Add pose-based features such as squat counter, arm raise counter, or exercise form checking.
- Add unit tests for the rule-based gesture recognition logic.
- Add sample images and output screenshots to the repository.
- Deploy the application using Streamlit Community Cloud or another hosting platform.

6. Conclusion

MediaPipe Vision Studio demonstrates how pre-trained computer vision tools can be combined into a simple and interactive web application. The project does not aim to build a production-level vision system. Instead, it focuses on making core computer vision concepts practical, visual, and easy to understand through a lightweight Streamlit dashboard.

The current implementation is suitable for a beginner-level educational project. It provides a clear demonstration of multiple computer vision tasks while leaving room for future improvements in real-time processing, gesture accuracy, testing, and deployment.